

OpenSSH

Stéphane Gill

Stephane.Gill@CollegeAhuntsic.qc.ca

Table des matières

Introduction.....	2
Le protocole SSH.....	2
Fonctionnement de SSH.....	2
Mise en place du canal sécurisé.....	3
L'authentification	3
Démarrage de sshd.....	3
Configuration de OpenSSH	4
Utilisation de ssh pour les connexions à distance.....	5
Retransmission X11.....	6
Utilisation de scp pour copier des fichiers.....	6
Utilisation de sftp pour remplacer ftp.....	7
Retransmission de ports.....	7
Utiliser ssh sans mot de passe.....	8
Configuration de la source.....	8
Configuration de la cible.....	8
Accès à un système de fichier via ssh.....	8
Installation.....	8
Utilisation.....	9
Références.....	9

Document écrit par Stéphane Gill

© Copyright 2005 Stéphane Gill

Ce document est soumis à la licence GNU FDL. Permission vous est donnée de distribuer et/ou modifier des copies de ce document tant que cette note apparaît clairement.

Introduction

Internet permet de réaliser un grand nombre d'opérations à distance, dont l'administration de serveurs. Le protocole Telnet ou les r-commandes (*rsh*, *rlogin* et *rexec*) ont toujours été utilisés pour cette tâche. L'inconvénient majeur de ces commandes est de faire circuler en clair sur le réseau les informations échangées, notamment l'identifiant (*login*) et le mot de passe pour l'accès à la machine distante. L'utilisation de tels protocoles rend donc vulnérable l'ordinateur distant en facilitant l'obtention du mot de passe de l'administrateur.

Une solution à ce problème est de recourir au protocole SSH (Secure Shell) qui répond à cette problématique et permet d'accéder à un ordinateur distant à travers une communication chiffrée.

Le protocole SSH

Le protocole SSH est un protocole permettant à un utilisateur d'ouvrir un terminal sur un ordinateur distant ou de transférer des fichiers de manière sécurisée. Ce protocole fait en sorte que les données circulant entre le client et le serveur soient encryptées, garantissant ainsi leur confidentialité.

OpenSSH est une mise en application OpenSource du protocole SSH qui fournit une série d'utilitaires permettant de remplacer les commandes *telnet*, *ftp*, *rlogin*, *rsh*.

Fonctionnement de SSH

L'établissement d'une connexion SSH se fait en deux étapes :

- Le serveur et le client s'identifient mutuellement afin de mettre en place un canal sécurisé.
- Le client s'authentifie auprès du serveur pour obtenir une session.

Mise en place du canal sécurisé

La mise en place du canal sécurisé débute par une phase de négociation entre le client et le serveur afin de s'entendre sur la méthode d'encryptage à utiliser. En effet le protocole SSH est prévu pour fonctionner avec un grand nombre d'algorithmes d'encryptage, c'est pourquoi le client et le serveur doivent dans un premier temps échanger les algorithmes qu'ils supportent.

Ensuite, afin d'établir une connexion sécurisée, le serveur envoie sa clé publique d'hôte au client tandis que le client envoie au serveur la clé de session. À partir de cet instant les communications sont encryptées grâce à un algorithme d'encryptage symétrique qui utilise la clé de session partagée par le client et le serveur.

L'authentification

Une fois que la connexion sécurisée est mise en place entre le client et le serveur, le client doit s'identifier sur le serveur afin d'obtenir un droit d'accès. Il existe deux méthodes :

- Par nom d'utilisateur et un mot de passe.
- Par clé publique et clé privée.

La méthode d'authentification par clés sera décrite dans la section « Utilisation de ssh sans mot de passe »

Démarrage de sshd

Sous CentOS, *ssh* est maintenant toujours installé. Il est possible de démarrer/arrêter/redémarrer le démon *ssh* à l'aide de la commande *service* :

```
# service sshd start
# service sshd stop
# service sshd restart
```

Pour s'assurer que *ssh* est disponible au démarrage du serveur, utiliser la commande suivante:

```
chkconfig -- level 35 sshd on
```

La vérification du fonctionnement du démon *ssh* peut s'effectuer à l'aide de la commande *pgrep*:

```
pgrep -l sshd
```

Configuration de OpenSSH

Le fichier de configuration de OpenSSH est `/etc/ssh/sshd_config`, en voici un exemple:

```
# The strategy used for options in the default sshd_config shipped
# with OpenSSH is to specify options with their default value where
# possible, but leave them commented. Uncommented options change a
# default value.

#Port 22
#Protocol 2,1
#ListenAddress 0.0.0.0
#ListenAddress ::

# HostKey for protocol version 1
#HostKey /etc/ssh/ssh_host_key
# HostKeys for protocol version 2
#HostKey /etc/ssh/ssh_host_rsa_key
#HostKey /etc/ssh/ssh_host_dsa_key

# Lifetime and size of ephemeral version 1 server key
#KeyRegenerationInterval 3600
#ServerKeyBits 768

# Logging
#obsoletes QuietMode and FascistLogging
#SyslogFacility AUTH
SyslogFacility AUTHPRIV
#LogLevel INFO

# Authentication:

#LoginGraceTime 120
#PermitRootLogin yes
#StrictModes yes

#RSAAuthentication yes
#PubkeyAuthentication yes
#AuthorizedKeysFile      .ssh/authorized_keys

# rhosts authentication should not be used
#RhostsAuthentication no
# Don't read the user's ~/.rhosts and ~/.shosts files
#IgnoreRhosts yes
# For this to work you will also need host keys in
/etc/ssh/ssh_known_hosts
#RhostsRSAAuthentication no
# similar for protocol version 2
#HostbasedAuthentication no
# Change to yes if you don't trust ~/.ssh/known_hosts for
# RhostsRSAAuthentication and HostbasedAuthentication
```

```
#IgnoreUserKnownHosts no

# To disable tunneled clear text passwords, change to no here!
#PasswordAuthentication yes
#PermitEmptyPasswords no

# Change to no to disable s/key passwords
#ChallengeResponseAuthentication yes

# Kerberos options
#KerberosAuthentication no
#KerberosOrLocalPasswd yes
#KerberosTicketCleanup yes

#AFSTokenPassing no

# Kerberos TGT Passing only works with the AFS kaserver
#KerberosTgtPassing no

# Set this to 'yes' to enable PAM keyboard-interactive authentication
# Warning: enabling this may bypass the setting of
# 'PasswordAuthentication'
#PAMAuthenticationViaKbdInt no

#X11Forwarding no
X11Forwarding yes
#X11DisplayOffset 10
#X11UseLocalhost yes
#PrintMotd yes
#PrintLastLog yes
#KeepAlive yes
#UseLogin no
#UsePrivilegeSeparation yes
#PermitUserEnvironment no
#Compression yes

MaxStartups 20
# no default banner path
#Banner /some/path
#VerifyReverseMapping no

# override default of no subsystems
Subsystem      sftp      /usr/libexec/openssh/sftp-server
```

Utilisation de *ssh* pour les connexions à distance

La commande *ssh* est un substitut sécurisé des commandes *rlogin*, *rsh* ou *telnet*. Elle permet d'ouvrir une session sur un ordinateur distant et d'y exécuter des commandes.

Par exemple, pour vous connecter à un ordinateur distant appelé *pingouin.exemple.net*, entrez la commande suivante à l'invite de l'interpréteur de commande :

```
ssh root@pingouin.exemple.net
```

La première fois qu'une connexion à un ordinateur distant à l'aide de *ssh* est demandé, le système affiche un message semblable à celui-ci :

```
The authenticity of host 'pingouin.exemple.net' can't be established.  
DSA key fingerprint is  
94:68:3a:3a:bc:f3:9a:9b:01:5d:b3:07:38:e2:11:0c.  
Are you sure you want to continue connecting (yes/no)?
```

La commande *ssh* peut aussi être utilisée pour exécuter une commande sur l'ordinateur distant sans vous connecter à un terminal. Par exemple, si vous souhaitez exécuter la commande *ls /usr/share/doc* sur l'ordinateur distant *pingouin.exemple.net*, entrez la commande suivante à l'invite du shell :

```
ssh pingouin.exemple.net ls /usr/share/doc
```

Retransmission X11

Lorsque l'option *X11Forwarding* est activée dans le fichier de configuration, l'ouverture d'une session X11 via une connexion *ssh* est aussi facile que l'exécution d'un programme X sur un ordinateur local. Lorsqu'un programme X est exécuté à partir d'une console *ssh*, le client et le serveur *ssh* créent un nouveau canal sécurisé et les données du programme X sont ensuite envoyées à l'ordinateur client de façon transparente.

Utilisation de *scp* pour copier des fichiers

La commande *scp* permet de transférer des fichiers entre des ordinateurs au moyen d'une connexion sécurisée. Par exemple, pour transférer le fichier local *shadowman* vers votre compte dans *pingouin.exemple.net*, la commande suivante doit être tapée :

```
scp shadowman utilisateur@pingouin.exemple.net:/home/utilisateur
```

Il est également possible de transférer un fichier distant sur le système local. Par exemple, pour transférer le contenu du répertoire */downloads* vers un répertoire existant nommé *uploads* sur l'ordinateur distant *pingouin.exemple.net*, entrez ce qui suit à l'invite du shell :

```
scp /downloads/* nom-d'utilisateur@pingouin.exemple.net:/uploads/
```

Utilisation de *sftp* pour remplacer *ftp*

L'utilitaire *sftp* peut être utilisé pour ouvrir une session *ftp* interactive. Cet utilitaire est semblable à *ftp* mais il utilise une connexion sécurisée. La syntaxe générale de cet utilitaire est :

```
sftp nom-d'utilisateur@nom-d'hôte.com
```

Une fois authentifié, vous pouvez utiliser un ensemble de commandes semblables à celles de FTP. Pour plus d'information sur la une liste des commandes de *sftp* consultez la page de manuel à l'aide de la commande :

```
man sftp
```

Retransmission de ports

Il est possible de sécuriser des protocoles TCP/IP non-sécurisés en utilisant la retransmission de ports (port forwarding) à l'aide de *ssh* pour créer un tunnel. La retransmission de ports consiste à associer un port local du client vers un port distant du serveur. Pour créer un canal de retransmission de port, utiliser la commande suivante:

```
ssh -L port_local:hôte_distant:port_distant login@hôte_distant
```

Exemple:

Vérifier le courrier électronique sur le serveur *poste.ahuntsic.qc.ca* au moyen du protocole POP à travers une connexion encryptée.

```
ssh -L 1100 :poste.ahuntsic.qc.ca :110 poste.ahuntsic.qc.ca
```

Une fois le canal de retransmission de ports en place, le client de mail devra interroger l'ordinateur local sur le port 1100.

Utiliser *ssh* sans mot de passe

Il est possible d'utiliser *ssh*, *scp* et *sftp* sans avoir à entrer de mot de passe. Pour ce faire, il faut créer une paire de clés d'autorisation. Les clés créées sont propres aux utilisateurs, elles doivent donc être créées pour chacun des utilisateurs.

Configuration de la source

- Générer une paire de clés :

```
ssh-keygen -t dsa
```

La paire de clés est stockée dans le répertoire `.ssh` de l'utilisateur. Le fichier `id_dsa` contient la clé privée et le fichier `id_dsa.pub` la clé publique.

Copier la clé publique sur l'ordinateur cible dans le compte de l'utilisateur.

Configuration de la cible

Ajouter la clé publique dans le fichier `~/.ssh/authorized_keys` :

```
cat public_key.tmp >> ~/.ssh/authorized_keys
```

Accès à un système de fichier via *ssh*

Shfs (*Shell File System*) est un module du noyau Linux qui permet de rendre disponible un système de fichiers distants via *ssh*. Le principal avantage de *shfs* est de pouvoir fournir l'accès à un système de fichiers distants via un canal de communications encryptés et ce, sans avoir à installer sur le serveur de fichiers autre chose qu'un serveur *ssh*.

Installation

L'installation de *shfs* s'effectue uniquement sur le client. Avant d'installer *shfs*, il faut s'assurer que :

- le client *ssh* soit installé;

- les sources du noyau 2.4.10 ou plus récent soit installées.

De façon générale, il faut télécharger les sources de *shfs* et les compiler :

```
tar -xvzf shfs-0.33.tar.gz
cd shfs-0.33
make
make install
modprobe shfs
```

Il est aussi possible de trouver des paquetages *rpm* de *shfs* pour certaines versions de *Fedora*, ce qui simplifie grandement l'installation.

Utilisation

L'accès au système de fichiers distants peut se faire grâce aux commandes *shfsmount* ou *mount*.

Exemple :

Monter le répertoire de l'utilisateur *greg* qui se trouve sur le serveur *linux1* dans le répertoire */mnt/shfs*

```
shfsmount greg@linux1 /mnt/shfs
```

ou

```
mount -t shfs greg@linux1 /mnt/shfs
```

Références

Aron Hsiao, « Sécurité sous Linux », Campus press, 2001.

« Red Hat Linux 9 : Red Hat Linux Security Guide », Red Hat Inc., 2002.

« Red Hat Linux 9 : Guide de référence Red Hat ». Red Hat Inc., 2003.

Greg Charot, « SHFS ou comment monter des partages via SSH », supinfo, 2004.